Explicación Detallada de la Aplicación de Gestión de Tareas

1. Introducción

1.1 Propósito de la Aplicación

La aplicación "Todo App" es un sistema de gestión de tareas personales desarrollado en Python que permite a los usuarios organizar, priorizar y hacer seguimiento de sus actividades diarias. El propósito principal de esta aplicación es proporcionar una herramienta robusta y fácil de usar que ayude a los usuarios a mejorar su productividad mediante la organización sistemática de sus tareas.

Los objetivos fundamentales que persigue esta aplicación son:

1. **Organización eficiente**: Permitir la categorización de tareas mediante categorías principales y subcategorías específicas
2. **Priorización inteligente**: Sistema de prioridades (Alta, Media, Baja) para gestionar tareas según su urgencia
3. **Persistencia de datos**: Guardado confiable en formato JSON que permite portabilidad y legibilidad
4. **Búsqueda avanzada**: Funcionalidad de búsqueda recursiva que permite encontrar tareas por múltiples criterios
5. **Análisis estadístico**: Generación de informes que proporcionan insights sobre el estado de las tareas
6. **Experiencia de usuario intuitiva**: Menús claros y navegación sencilla mediante sistema de submenús

2. Análisis del Diagrama de Flujo

Este diagrama representa un **sistema de gestión de tareas** con las siguientes fases:

1. Inicio y Configuración

- Inicializa variables (continuar=True, tareas_cargadas=False, guardado=True)
- Entra en bucle principal de ejecución

2. Menú Principal

- Solicita opción al usuario
- Decide el flujo según la selección

3. Rutas Principales:

A. Cargar/Guardar Tareas

- Carga tareas existentes desde archivo
- Guarda tareas con confirmación
- Busca tareas específicas por ID

B. Gestión de Tareas

- **Listar**: Muestra todas las tareas o marca como completada
- **Agregar**: Crea nueva tarea con descripción, prioridad, categoría y fecha límite
- **Editar**: Modifica campos de tarea existente
- **Eliminar**: Borra tarea por ID

C. Funciones Especiales

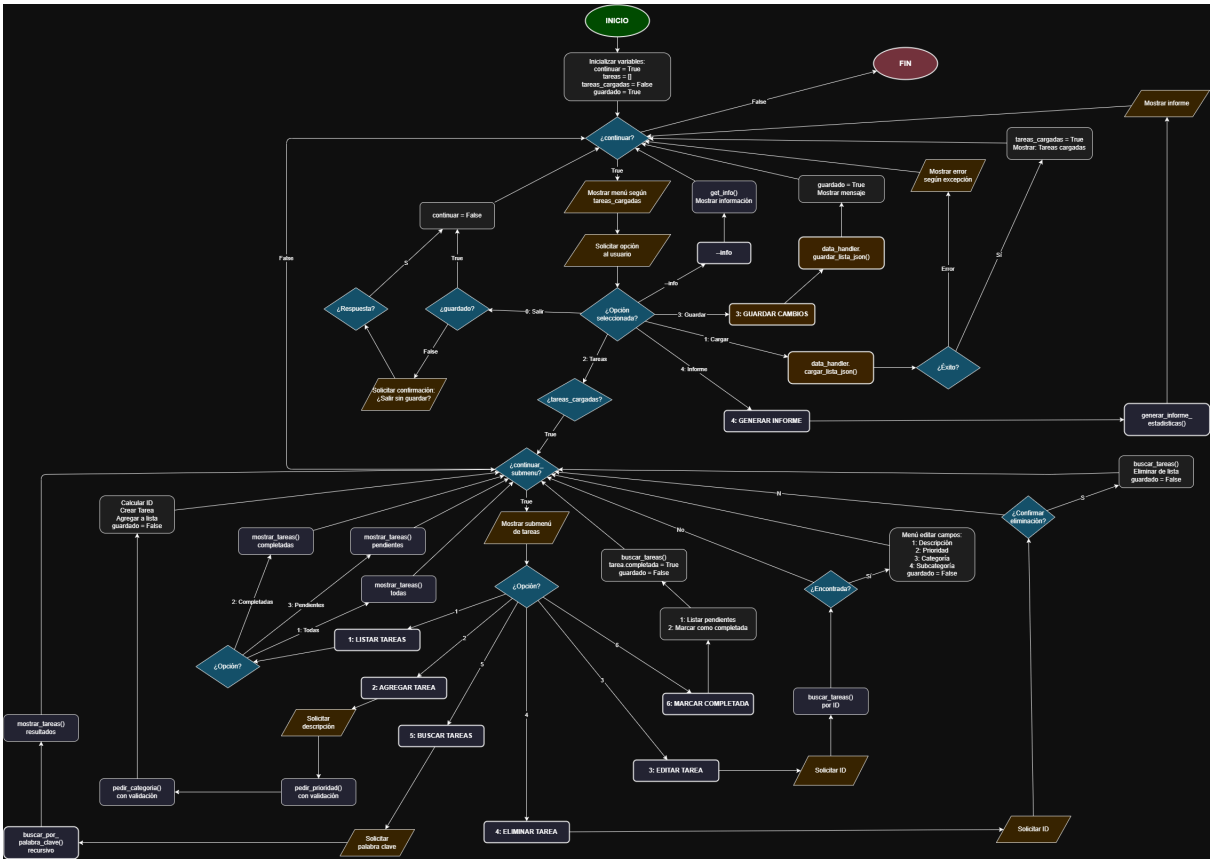
- **Generar Informe**: Crea estadísticas del estado de tareas

4. Control de Cambios

- Detecta si hay tareas sin guardar
- Solicita confirmación antes de salir si hay cambios pendientes

5. Finalización

- Genera informe final
- Muestra mensaje de cierre



3. Descripción Detallada del Código

3.1 Módulo de Entidades (`entities.py`)

Clase Tarea con `@dataclass`:

```
@dataclass
class Tarea:
    id: int
    descripcion: str
    prioridad: Prioridad
    completada: bool
    categoria: Categoria
```

Genera automáticamente `__init__`, `__repr__` y `__eq__`.

Enumeraciones:

```
class Prioridad(Enum):
    ALTA = "Alta"
```

```
    MEDIA = "Media"
    BAJA = "Baja"
```

Garantizan valores válidos, autocompletado en IDEs y refactoring seguro.

Mapeo de Subcategorías:
Evita combinaciones inválidas (ej: "Trabajo → Ejercicio") asociando cada categoría principal con sus subcategorías válidas.

```
SUBCATEGORIAS_POR_CATEGORIA = {
    CategoriaPrincipal.TRABAJO: SubcategoriaTrabajo,
    CategoriaPrincipal.PERSONAL: SubcategoriaPersonal,
    CategoriaPrincipal.ESTUDIO: SubcategoriaEstudio
    }
```

3.2 Función Recursiva de Búsqueda

```
def buscar_por_palabra_clave(estructura, categoria_buscada, tarea=None)
```

Busca tareas recursivamente procesando diferentes estructuras:

- **Lista**: Divide en primer elemento y resto, procesa ambos
- **Tarea**: Busca en prioridad y descripción, luego recursivamente en su categoría
- **Categoria**: Compara con categoría principal, subcategoría o texto
- **Lista vacía**: Caso base, retorna `[]`

Combina todos los resultados encontrados.

3.3 Gestión de IDs Automáticos

```
id = max([t.id for t in tareas], default=0) + 1
```

Extrae todos los IDs, encuentra el máximo y suma 1. No requiere contador global.

3.4 Sistema de Validación

```
def pedir_prioridad():
    opciones = [p.value for p in Prioridad]
    while True:
        entrada = input("Prioridad: ").strip().capitalize()
        if entrada in opciones:
            return Prioridad(entrada)
```

- Genera opciones dinámicamente desde el Enum
- Bucle infinito hasta entrada válida
- Normaliza entrada (`.strip()`, `.capitalize()`)
- Patrón reutilizado en categorías y subcategorías

4. Manejo de Errores y Robustez

4.1 Carga de Datos

```
try:
    tareas = data_handler.cargar_lista_json()
except FileNotFoundError as e:
    print(f"No se pudo cargar: {e}")
except ValueError as e:
    print(f"Formato inválido: {e}")
except Exception as e:
    print(f"Error inesperado: {e}")
```

Jerarquía de excepciones:

- **FileNotFoundError**: Archivo no existe
- **ValueError**: JSON malformado o datos inválidos
- **Exception**: Captura errores no anticipados

Previene crash total y no permite crear tareas aunque si no existe archivo.

4.2 Validación de IDs

```
try:
    tarea = utils.buscar_tareas(tareas, id)
```

```
except IndexError:
    print(f"No se encontró tarea con ID {id}")
```

Captura IDs inexistentes. Mejora sugerida: validar que ID sea numérico capturando `ValueError` en conversión `int(id)`.

4.3 Protección contra Pérdida de Datos

```
if not guardado:
    opcion = input("¿Salir sin guardar? S/N: ")
    if opcion.upper() == "S":
        continuar = False
```

Doble barrera:

- Flag `guardado` detecta cambios pendientes
- Confirmación explícita del usuario con validación S/N
- Normaliza entrada con `.upper()`

4.4 Listas Vacías

```
if not lista:
    print("No hay tareas para mostrar.")
    return
```

Evita situación confusa mostrando mensaje claro.

5. Arquitectura y Separación de Responsabilidades

5.1 Arquitectura en Capas

La aplicación sigue un patrón de arquitectura de tres capas:

```
┌──────────────────────────────────────────┐
│  Capa de Presentación (__main__.py)  │
│  - Interfaz de usuario              │
│  - Menús y navegación               │
│  - Captura de entrada               │
└──────────────────────────────────────────┘
                    │
```

```
            │
            ▼
┌──────────────────────────────────────┐
│  Capa de Lógica de Negocio           │
│  (bussiness/utils.py, reports.py)    │
│  - Validaciones                      │
│  - Búsquedas                         │
│  - Generación de informes            │
└──────────────────────────────────────┘
            │
            ▼
┌──────────────────────────────────────┐
│  Capa de Datos                       │
│  (data/data_handler.py, entities.py) │
│  - Persistencia                      │
│  - Modelos de datos                  │
│  - Serialización/Deserialización     │
└──────────────────────────────────────┘
```

Ventajas de esta arquitectura:

1. **Bajo acoplamiento**: Cambiar la UI no afecta la lógica de negocio
2. **Alta cohesión**: Cada módulo tiene responsabilidades claras
3. **Testabilidad**: Cada capa puede probarse independientemente
4. **Escalabilidad**: Fácil añadir nuevas funcionalidades

5.2 Módulo data_handler.py: Encapsulación de Persistencia

```
# data_handler.py
def cargar_lista_json():
    # Lógica de carga

def guardar_lista_json(tareas):
    # Lógica de guardado
```

Beneficios:

- **Abstracción**: La app no conoce el formato (JSON)
- **Punto único de cambio**: Modificar almacenamiento solo aquí
- **Reutilización**: No duplicación de lógica

Escalabilidad futura: Fácil migrar a SQLite sin tocar `__main__.py`

6. Pruebas y Casos Edge

6.1 Test de Creación de Entidades

Resultado: Valida que `@dataclass` asigna correctamente todos los campos (id, descripción, prioridad, completada, categoría).

Caso edge cubierto: Verifica que estructuras anidadas (Categoria) mantienen integridad de tipos y valores.

6.2 Test de Asignación de IDs

Resultado: Confirma que el ID asignado es `max(ids) + 1`.

Caso edge cubierto: Lista vacía retorna ID=1 gracias a `default=0`. IDs no consecutivos (tras eliminación) funcionan correctamente.

6.3 Test de Búsqueda Exitosa

Resultado: `buscar_tareas("2")` retorna tarea con ID=2 y descripción correcta.

Caso edge cubierto: Conversión automática string→int simula entrada real de usuario desde `input()`.

6.4 Test de Búsqueda Fallida

Resultado: `buscar_tareas("999")` lanza `IndexError`.

Caso edge cubierto: ID inexistente genera excepción controlada en lugar de retornar `None` o crash silencioso.

6.5 Test de Búsqueda Recursiva

Resultado: Encuentra tareas por substring en descripción ("Meet" → "Meeting") y por categoría ("TRABAJO").

Caso edge cubierto: Búsqueda case-insensitive y navegación en estructuras anidadas (Tarea → Categoria → principal).

6.6 Test de Cambio de Estado

Resultado: Marcar tarea como completada cambia flag de `False` a `True` y aparece en filtro de completadas.

Caso edge cubierto: Validación de mutabilidad por referencia y persistencia del cambio en la lista original.

6.7 Test de Generación de Informe

Resultado: `generar_informe_estadisticas()` retorna string no vacío.

Caso edge cubierto: Informe con lista vacía retornaría mensaje válido sin crash.

6.8 Test de Guardado con Mocking

Resultado: `guardar_lista_json()` se invoca exactamente una vez con la lista correcta.

Caso edge cubierto: Verifica identidad del objeto (`assertIs`) sin I/O real, evitando fallos por permisos o espacio en disco.

6.9 Test de Carga con Mocking

Resultado: `cargar_lista_json()` retorna datos simulados correctamente.

Caso edge cubierto: Simula carga exitosa con datos predecibles sin depender de archivo físico.

6.10 Test de Propagación de Errores

Resultado: `cargar_lista_json()` con JSON corrupto propaga `ValueError`.

Caso edge cubierto: Errores no se silencian, llegan al caller con stack trace completo para debugging efectivo.

8. Conclusión

8.1 Logros

Sistema CRUD completo con persistencia JSON, búsqueda recursiva avanzada, categorización multinivel con Enums, generación de informes y suite de pruebas con `unittest` .

8.2 Lecciones Aprendidas

- Recursión en estructuras heterogéneas con pattern matching
- Type-safety mediante Enums y dataclasses
- Arquitectura en capas facilita mantenimiento

- Mocking para tests aislados

8.4 Mejoras Futuras

Migración a SQLite, fechas de vencimiento, etiquetas flexibles, recordatorios, exportación CSV.